

Programmazione II e ingegneria del software (2024/2025)

Modulo: Programmazione II

Docente: **Prof. Fausto Spoto**

Università degli Studi di Verona

Anno Accademico 2024/2025

Indice

Programmazione II - Introduzione	2
Lezione del 2/10/2024, 13:30 - 14:30, aula Magna	2
Lezione del 3/10/2024, ore 16:30 - 18:30, aula Magna	2
Laboratorio del 4/10/2024, 12:30 - 14:30, aula Delta	4
Lezione del 4/10/2024, ore 17:30 - 18:30, aula Tessari	5
Lezione del 9/10/2024, ore 13:30 - 14:30, aula Magna	5
Lezione del 10/10/2024, ore 16:00 - 18:00, aula Magna	5
Laboratorio del 11/10/2024, ore 12:30 - 14:30, aula Delta	6
Lezione dell'11/10/2024, ore 14:30 - 15:30, aula Delta	6
Lezione del 16/10/2024, ore 13:30 - 14:30, aula Magna	6
Lezione del 17/10/2024, ore 13:30 - 16:30, aula Magna	7
Laboratorio del 18/10/2024, ore 12:30 - 14:30, aula Delta	7
Lezione del 23/10/2024, ore 13:30 - 14:30, aula Magna	8
Lezione del 24/10/2024, ore 16:30 - 18:30, aula Magna	9
Laboratorio del 25/10/2024, ore 12:30 - 14:30, aula Delta	9
Lezione del 30/10/2024, ore 13:30 - 14:30, aula Magna	10
Lezione del 31/10/2024, ore 16:30 - 18:30, aula Magna	10
Lezione del 6/11/2024, ore 13:30 - 14:30, aula Magna	11
Laboratorio dell'8/11/2024, ore 12:30 - 14:30, aula Delta	11
Lezione del 13/11/2024, ore 13:30 - 14:30, aula Magna	12
Lezione del 14/11/2024, ore 13:30 - 15:30, aula Magna	13
Laboratorio del 15/11/2024, ore 12:30 - 14:30, aula Delta	14
Lezione del 20/11/2024, ore 13:30 - 14:30, aula Magna	16
Lezione del 21/11/2024, ore 16:30 - 18:30, aula Magna	16
Laboratorio del 22/11/2024, ore 12:30 - 14:30, aula Delta	18
Lezione del 4/12/2024, ore 13:30 - 14:30, aula Magna	19
Lezione del 5/12/2024, ore 16:30 - 18:30, aula Magna	21
Laboratorio del 6/12/2024, ore 12:30 - 14:30, aula Delta	22
Lezione dell'11/12/2024, ore 13:30 - 14:30, aula Magna	23
Lezione del 12/12/2024, ore 16:30 - 18:30, aula Magna	24
Laboratorio del 13/12/2024, ore 12:30 - 14:30, aula Delta	25

Lezione del 18/12/2024, ore 13:30 - 14:30, aula Magna	25
Laboratorio del 18/12/2024, ore 16:30 - 18:30, aula Delta	25
Lezione del 19/12/2024, ore 16:30 - 18:30, aula Magna	26
Laboratorio del 20/12/2024, ore 12:30 - 14:30, aula Delta	27
Laboratorio del 10/1/2025, ore 12:30 - 14:30, aula Delta	28
Laboratorio del 17/1/2025, ore 12:30 - 14:30, aula Delta	28
Laboratorio del 24/1/2025, ore 12:30 - 14:30, aula Delta	29
Primo Appello di Programmazione 2 del 5/2/2025, aula Delta	29
Secondo Appello di Programmazione 2 del 20/2/2025, aula Delta	29

Programmazione II - Introduzione

La parte di Programmazione 2 verrà effettuata in presenza, sia per la parte di teoria che per la parte di laboratorio.

Per lavorare da casa, gli studenti sono invitati a installare Eclipse (o equivalente) sul proprio computer, con supporto per Java almeno versione 9, in modo da potere effettuare da casa gli esercizi del laboratorio.

L'esame della parte di Programmazione 2 consiste nella scrittura o modifica di programmi Java, al calcolatore. Gli esami verranno effettuati sui computer dei laboratori didattici dell'Università. Durante l'esame è possibile consultare materiale didattico, appunti e libri, ma internet non sarà disponibile. Programmi che non compilano non saranno valutati. Il voto complessivo sarà la media del voto di Programmazione 2 e del voto di Ingegneria del Software.

Chi fosse impossibilitato a seguire le lezioni in presenza può fare riferimento alle [registrazioni delle lezioni passate](#).

 [Compiti degli anni passati con soluzione](#)

Lezione del 2/10/2024, 13:30 - 14:30, aula Magna

Introduzione al corso. Storia di Java. Bytecode. JDK e JRE. Concetto di programmazione ad oggetti: ereditarietà, incapsulazione e polimorfismo. Compilazione ed esecuzione da linea di comando. Primo esempio di programma Java. Istruzioni di stampa. Compilazione ed esecuzione dello stesso programma usando l'ambiente Eclipse. Tipi primitivi (`boolean`, `char`, `byte`, `short`, `int`, `long`, `float` e `double`). Dichiarazione di variabili e costanti (`final`). Inizializzazione obbligatoria delle variabili locali prima di venire lette. Operazioni aritmetiche. Operazioni aritmetiche ibride. Assegnamento. Assegnamento composto. Pre e post incremento/decremento.

Esempi

-  [Hello.java](#)
-  [Variabili.java](#)

 [Download folder](#)

Lezione del 3/10/2024, ore 16:30 - 18:30, aula Magna

Lettura dell'input da tastiera tramite classe `java.util.Scanner`. Importazione delle classi di libreria (`java.lang` è sempre importato per default). Istruzioni `if`, `while`, `do`, `for` e `switch`: quasi identiche a quelle del linguaggio C. Identità delle stringhe (`==`) e uguaglianza delle stringhe (`equals`).

ghe (`equals()`). Operatore di concatenazione fra stringhe `+` e conseguente uso implicito dei metodi `concat()` e `valueOf()` di `java.lang.String`. Concetto di classe e oggetto. Creazione di un oggetto dalla sua classe con l'operatore `new`. Il caso speciale dei letterali di tipo `java.lang.String`. Concetto di metodo: inviato a un oggetto (metodo d'istanza) o inviato a una classe (metodo statico). Sintassi della chiamata di metodo. Garbage e garbage collection.

Metodi di uso frequente della classe di libreria `java.lang.String` (classe immutabile, nel senso che i suoi oggetti non sono più modificabili dopo la creazione)

- `String(String other)` (costruttore di copia: crea un clone)
- `char charAt(int index)`
- `int compareTo(String other)` (ritorna negativo, zero, oppure positivo)
- `int compareToIgnoreCase(String other)` (ritorna negativo, zero, oppure positivo)
- `String concat(String other)` (implicitamente usato per la concatenazione con `+`)
- `boolean endsWith(String end)`
- `boolean equals(Object other)`
- `boolean equalsIgnoreCase(String other)`
- `static String format(String format, Object... args)`
- `int indexOf(int character)`
- `int indexOf(String what)`
- `boolean isEmpty()`
- `int length()`
- `boolean startsWith(String what)`
- `String substring(int start)` (da `start` incluso)
- `String substring(int start, int end)` (da `start` incluso ad `end` escluso)
- `String toLowerCase()`
- `String toUpperCase()`
- `String trim()`
- `static String valueOf(int i)` (esegue una conversione esplicita di tipo; esiste per tutti i tipi primitivi, non solo per `int`; implicitamente usato per la concatenazione con `+`)

Metodi di uso frequente della classe di libreria `java.lang.System`

- `err` (costante della classe che fa riferimento allo standard error)
- `in` (costante della classe che fa riferimento allo standard input)
- `out` (costante della classe che fa riferimento allo standard output)
- `static long currentTimeMillis()` (ritorna il numero di millisecondi passati dalla mezzanotte dell'1 gennaio 1970 UTC)

Metodi di uso frequente della classe di libreria `java.util.Scanner`

- `Scanner(source)` (costruttore, che crea uno `Scanner` legato alla sorgente indicata)
- `void close()` (chiude lo `Scanner`: dopo non può più essere usato)
- `double nextDouble()`
- `float nextFloat()`
- `int nextInt()`
- `String nextLine()`
- `long nextLong()`

Esempi

-  `Alfabeto.java`
-  `Curioso.java`
-  `Pappagallo.java`

 Download folder



Laboratorio del 4/10/2024, 12:30 - 14:30, aula Delta

Metodi di uso frequente della classe di libreria `java.util.Random`

- `boolean nextBoolean()`
- `double nextDouble()`
- `float nextFloat()`
- `int nextInt()`
- `int nextInt(int max)` (restituisce un numero casuale tra 0 e `max` escluso)
- `long nextLong()`

Metodi di uso frequente della classe di libreria `java.lang.Math`

- `static double E` (costante della classe)
- `static double PI` (costante della classe)
- `static int abs(int i)` (esiste anche per altri tipi numerici)
- `static double cos(double d)`
- `static double log(double d)` (in base e)
- `static double log10(double d)` (in base 10)
- `static int max(int a, int b)` (esiste anche per altri tipi numerici)
- `static int min(int a, int b)` (esiste anche per altri tipi numerici)
- `static double sin(double d)`
- `static double sqrt(double d)`
- `static double tan(double d)`
- `static double toDegrees(double radians)`
- `static double toRadians(double degrees)`

Esempi di utilizzo delle classi `java.lang.System`, `java.util.Random` e `java.lang.Math`.

1. Si scriva un programma che legge da tastiera un `long` non negativo `n` e lo richiede a oltranza se l'utente lo inserisse negativo. Quindi genera `n` coppie `(x,y)` fatte da due numeri casuali di tipo `double`, fra -1 e 1. Per ogni coppia controlla se la coordinata `(x,y)` sta dentro il cerchio di raggio 1 centrato sull'origine degli assi e in tal caso incrementa una variabile `dentro` di tipo `long`. Alla fine stampa il valore della formula `dentro * 4 / n` senza perdere le cifre che seguono la virgola.
2. Si scriva un programma Java che legge un intero non negativo `n` da tastiera e stampa una cornice `n x n`:

```
@@@@@
@      @
@      @
@      @
@      @
@@@@@
```

3. Si scriva un programma che stampa, una dopo l'altra, nove cornici, da quella `2 x 2` a quella `10 x 10`.
4. Si scriva un programma che stampa 26 volte la cornice `10 x 10`, usando ogni volta un carattere alfabetico diverso al posto di '@', dalla 'a' alla 'z'.
5. Si scriva un programma che legge `n >= 1` da tastiera e stampa una piramide di altezza `n`. Per esempio, per `n = 4` deve stampare:

```
  @
 @@
Undetermined error: @@@@
@@@@@
```

6. Si faccia la stessa cosa del punto precedente, ma stampando la piramide orizzontalmente:

```
  @
 @@
@@@
@@@@
@@@@
@@@@
 @
```

Esempi e soluzioni

 Download folder

-  [Casuali.java](#)
-  [Cerchio.java](#)
-  [Cornice.java](#)
-  [Cornice26.java](#)



Lezione del 4/10/2024, ore 17:30 - 18:30, aula Tessari

I due mondi dei tipi primitivi e dei tipi riferimento. Assenza di conversioni di tipo fra i due mondi (implicite o esplicite). I tipi primitivi non hanno metodi! Il valore speciale `null` per variabili di tipo riferimento. Mai chiamare metodi o accedere a `null`! Concatenazione di un oggetto a una stringa con l'operatore `+` e stampa di un oggetto con `System.out.println()` (e simili): conseguente chiamata implicita a `toString()`.

Esempi

 Download folder

-  [Cornice.java](#)
-  [Cornice2.java](#)
-  [Inizializzazione.java](#)
-  [Variabili.java](#)



Lezione del 9/10/2024, ore 13:30 - 14:30, aula Magna

Definizione di una nostra classe: campi, costruttore e metodi. Valore di default dei campi. Modificatori `public` e `private`. Incapsulazione dello stato degli oggetti. Parametro implicito `this`.

Esempi

 Download folder

-  [Date.java](#)
-  [MainDate.java](#)



Lezione del 10/10/2024, ore 16:00 - 18:00, aula Magna

Costruttore di default. Attivazione di un metodo o costruttore. Parametri formali e attuali. Record e stack di attivazione. Modificatore `final` per i campi e le variabili locali. Creazione di array per enumerazione degli elementi. Definizione di campi `static`. Controllo di legalità delle date. Overloading dei costruttori. Concatenazione dei costruttori con la sintassi `this()`.

Esempi

 Download folder

-  [Date.java](#)
-  [MainDate.java](#)

**Laboratorio del 11/10/2024, ore 12:30 - 14:30, aula Delta**

Si facciano i seguenti esercizi, senza usare funzioni di libreria che risolvono esattamente tali problemi

1. Si scriva un programma il cui main legge un intero non negativo da tastiera (se negativo, lo richiede ad oltranza) e lo trasforma in una stringa che rappresenta lo stesso numero in binario, che infine stampa. Per esempio, se viene immesso l'intero 123, deve costruire e poi stampare la stringa "1111011".
2. Si faccia la stessa cosa in ottale e in esadecimale.
3. Si faccia la stessa cosa in Base58, che è una numerazione le cui cifre sono le cifre arabe da 1 a 9, seguite dalle lettere alfabetiche maiuscole (tranne O e I) e poi dalle lettere alfabetiche minuscole (tranne la l). Si veda la descrizione [qui](#).
4. Si scriva un programma Java che legge una frase da tastiera e determina se tale frase è palindroma.

Soluzioni[Download folder](#)

- [Base58.java](#)
- [Binary.java](#)
- [Hexadecimal.java](#)
- [Hexadecimal2.java](#)
- [Octal.java](#)
- [Palindrome.java](#)

**Lezione dell'11/10/2024, ore 14:30 - 15:30, aula Delta**

Definizione automatizzata di un tipo enumerato tramite la parola chiave `enum`. Definizione di metodi di confronto fra oggetti.

Esempi[Download folder](#)

- [Date.java](#)
- [Language.java](#)
- [MainDate.java](#)

**Lezione del 16/10/2024, ore 13:30 - 14:30, aula Magna**

Enumerazione delle stagioni, metodo delle `Date` che ritorna la stagione di una data. Dichiarazione degli array, inizializzazione esplicita per enumerazione degli elementi oppure creazione con `new`. Lo pseudocampo `length` degli array. Ciclo `for each` sugli array. Creazione dell'array delle stagioni di un'array di date.

Metodi di uso frequente delle classi `E` definite tramite `enum`

- `static E[] values()` (ritorna l'array di tutti gli elementi dell'enumerazione)
- `static E valueOf(String name)` (ritorna l'elemento dell'enumerazione che ha il nome indicato)
- `int compareTo(E other)` (determina chi viene prima nell'enumerazione)
- `int ordinal()` (ritorna il numero d'ordine di un elemento dell'enumerazione)

Metodi di uso frequente della classe `java.util.Arrays`

- `static int binarySearch(int[] arr, int key)` (ritorna la posizione di `key` dentro `arr`, oppure un numero negativo se `arr` non contiene `key`. Assume che l'array `arr` sia ordinato. Questo metodo esiste anche per gli altri tipi primitivi numerici e per i tipi riferimento, nel qual

caso chiama `compareTo()` per decidere l'ordine)

- `static boolean equals(int[] arr1, int[] arr2)` (controlla che `arr1` e `arr2` abbiano stessa lunghezza e contengano gli stessi elementi nello stesso ordine. Questo metodo esiste anche per gli altri tipi primitivi nonché per array di tipi riferimento, nel qual caso chiama `equals()` fra tutte le coppie di oggetti da confrontare)
- `static void fill(int[] arr, int val)` (assegna `val` a tutti gli elementi di `arr`. Questo metodo esiste anche per tutti gli altri tipi primitivi e per array di tipi riferimento)
- `static void sort(int[] arr)` (ordina `arr` in senso crescente, in tempo $O(n \log n)$. Questo metodo esiste anche per tutti gli altri tipi primitivi numerici e per i tipi riferimento, nel qual caso chiama `compareTo()` per decidere l'ordine)
- `static String toString(int[] arr)` (ritorna una stringa che riporta gli elementi di `arr`, nel loro ordine. Questo metodo esiste anche per gli altri tipi primitivi e per array di tipi riferimento, nel qual caso chiama `toString()` sugli elementi dell'array e concatena il risultato.

Esempi

Download folder

- `Date.java`
- `Language.java`
- `MainDate.java`
- `Season.java`



Lezione del 17/10/2024, ore 13:30 - 16:30, aula Magna

Assegnamento fra variabili di tipo array. Array multidimensionali e loro struttura in memoria. Array sparsi. Definizione di metodi `static`. Esempio del contatore di oggetti creati di una classe. Esempio di un metodo che ritorna il primo gennaio di un anno e di un metodo che ritorna il 31 dicembre di un anno. Interfaccia pubblica di una classe. Classi e oggetti mutabili e immutabili. Aliasing fra variabili e rischi in caso di condivisione di oggetti mutabili. JavaDoc. Aggiunta dei commenti JavaDoc nel codice sorgente e generazione delle pagine HTML da Eclipse. Definizione di classi per estensione, ereditando campi e metodi. I costruttori non si ereditano. Notazione `super()` per la concatenazione dei costruttori da sottoclasse a superclasse. Esempio delle date generiche e poi italiane, americane e australiane.

Esempi

Download folder

- `AmericanDate.java`
- `Bidimensional.java`
- `Date.java`
- `ItalianDate.java`
- `MainDate.java`
- `MainInheritance.java`
- `Season.java`



Laboratorio del 18/10/2024, ore 12:30 - 14:30, aula Delta

1. Si vuole implementare una carta del gioco del poker, il cui valore è uno fra questi:

2 3 4 5 6 7 8 9 10 J Q K 1

Tale valore può essere visto come un numero fra 0 e 12. Una carta ha anche un seme (in inglese: suit) che può essere:

spades (♠), clubs (♣), diamonds (♦) e hearts (♥).

Si completi la seguente classe, che rappresenta una carta:

```
public class Card {
```

```

/**
 * Il valore della carta.
 */
private final int value;

/**
 * Il seme della carta.
 */
private final int suit;

/**
 * Genera una carta a caso con un valore da min (incluso) in su.
 *
 * @param min il valore minimo (0-12) della carta che può essere generata
 */
public Card(int min) { ... }

/**
 * Genera una carta a caso con un valore da 0 (incluso) in su.
 */
public Card() { ... }

public int getValue() { ...ritorna il valore della carta (0-12) }

public int getSuit() { ...ritorna il seme della carta (0-3) }

/**
 * Ritorna una descrizione della carta sotto forma di stringa, del tipo ♠10 oppure ♥J.
 */
public String toString() { ... }

/**
 * Determina se questa carta è uguale ad other.
 *
 * @param other l'altra carta con cui confrontarsi
 * @return true se e solo se le due carte sono uguali
 */
public boolean equals(Card other) { ... }
}

```

2. Si scriva una classe **Main** con un metodo iniziale **main** che crea una carta **card1** a caso, quindi crea ripetutamente una carta **card2** a caso finché non risulta che **card1** è **equals** con **card2**. A quel punto termina. Sia **card1** che tutte le **card2** dovranno venire stampate sul video man mano che vengono generate.
3. Si definiscano due enumerazioni **Value** e **Suit**, che rappresentano, rispettivamente, le 13 alternative per il valore delle carte e le quattro alternative per il loro seme. Si modifichi la classe **Card** in modo da usare queste enumerazioni al posto degli interi come valore e seme delle carte.
4. Si aggiunga un metodo **public int compareTo(Card other)** alla classe **Card**, che confronta una carta con un'altra e determina chi viene prima: le carte devono venire ordinate per valore; a parità di valore, devono venire ordinate per seme (picche, fiori, quadri, cuori).
5. Si generino le pagine JavaDoc da Eclipse.

Soluzione con gli int

 Download folder

-  Card.java
-  Main.java

Soluzione con gli enum

 Download folder

-  Card.java
-  Main.java
-  Suit.java
-  Value.java



Lezione del 23/10/2024, ore 13:30 - 14:30, aula Magna

Ridefinizione o aggiunta di metodi. Relazione di sottoclasse e superclasse. Tipo statico delle espressioni e tipo dinamico dei valori. Late binding dei metodi. Il class tag degli oggetti, usato per il late binding. Ereditarietà singola e multipla (Java ha solo ereditarietà singola).

Esempi

[Download folder](#)

- `AmericanDate.java`
- `AustralianDate.java`
- `Date.java`
- `ItalianDate.java`
- `MainInheritance.java`
- `Season.java`



Lezione del 24/10/2024, ore 16:30 - 18:30, aula Magna

La notazione `final` per metodi e classi. Visibilità `public`, `protected`, `package` e `private`. La classe `java.lang.Object` e i suoi metodi `toString()` ed `equals()`. Il cast fra tipi riferimento e sua differenza con la conversione di tipi primitivi. L'operatore `instanceof`. Ridefinizione del metodo `equals()` di `java.lang.Object`. Aggiunta di campi nelle sottoclassi. Richiamo dell'implementazione di un metodo della superclasse tramite la notazione `super.metodo(...)`. Esempio delle date e delle date con orario. Modifica conseguente ai metodi `equals()` e `compareTo()` delle date.

Esempi

[Download folder](#)

- `AmericanDate.java`
- `AustralianDate.java`
- `Date.java`
- `ItalianDate.java`
- `ItalianDateAndTime.java`
- `Main.java`
- `Season.java`



Laboratorio del 25/10/2024, ore 12:30 - 14:30, aula Delta

Si crei un package `it.univr.figures`. Al suo interno, si realizza il codice descritto sotto, in cui tutti i campi devono essere `private` per rispettare l'incapsulazione.

- Si scriva un'enumerazione `Color` che enumera cinque colori di vostra scelta, incluso il verde.
- Si scriva una classe `Figure` che rappresenta una figura geometrica colorata. Tale classe deve avere un costruttore che riceve un `Color`. Deve avere un metodo `double perimeter()` e un metodo `double area()`; fate ritornare ad entrambi 0. Inoltre deve avere un metodo `String toString()` che ritorna la stringa "area: A, perimeter: P, color: C", dove A è l'area della figura, P è il perimetro della figura e C è il colore della figura. Infine, deve avere un metodo accessore `protected` per il colore.
- Si scriva una sottoclasse `Rectangle` di `Figure` che rappresenta un rettangolo, con un costruttore che riceve colore, base e altezza `double` del rettangolo e in cui i metodi `double perimeter()` e `double area()` sono ridefiniti in modo da ritornare perimetro ed area del rettangolo, rispettivamente. Il metodo `String toString()` deve essere ridefinito in modo da ritornare la stringa "Rectangle of " seguita dalla chiamata al `toString()` della superclasse.
- Si scriva una sottoclasse `Circle` di `Figure` che rappresenta un cerchio, con un costruttore che riceve colore e raggio `double` del cerchio e in cui i metodi `double perimeter()` e `double area()` sono ridefiniti in modo da ritornare perimetro ed area del cerchio, rispettivamente. Il metodo `String toString()` deve essere ridefinito in modo da ritornare la stringa "Circle of " seguita dalla chiamata al `toString()` della superclasse.
- Si scriva una sottoclasse `Square` di `Rectangle` che rappresenta un quadrato, con un costruttore che riceve colore e lato `double` del quadrato. Non si ridefiniscano i metodi `double pe-`

rimeter() e `double area()`. Il metodo `String toString()` deve invece essere ridefinito in modo da ritornare la stringa "Square, a " seguita dalla chiamata al `toString()` della superclasse.

- Si scriva una sottoclasse `GreenDot` di `Circle` che rappresenta un cerchio di raggio 1 e colore verde, con un costruttore senza argomenti. Non si ridefinisca alcun metodo al suo interno.

Si scriva dentro `it.univr` una classe `MainFigures` con un metodo di partenza `main` che crea e stampa sul video un'istanza di ognuna delle figure geometriche implementate sopra. In tale classe è possibile chiamare il metodo `getColor()` sulle figure?

Esempio delle figure

 Download folder

-  `Circle.java`
-  `Color.java`
-  `Figure.java`
-  `GreenDot.java`
-  `MainFigures.java`
-  `Rectangle.java`
-  `Square.java`



Lezione del 30/10/2024, ore 13:30 - 14:30, aula Magna

Modifica del metodo `equals()` delle date:

- se due date hanno giorno, mese o anno diversi, non sono `equals`;
- altrimenti, se una data ha anche il tempo e l'altra non ha il tempo, non sono `equals`;
- altrimenti, se nessuna delle due date ha il tempo, sono `equals`;
- altrimenti, le due date sono `equals` se e solo se hanno lo stesso tempo (secondi, minuti e ore uguali).

Modifica del metodo `compareTo()` delle date:

- due date si mettono prima in ordine per giorno/mese/anno
- altrimenti, se una ha il tempo e l'altra non ha il tempo, si mette prima la data senza tempo
- altrimenti, se nessuna delle due date ha il tempo, si considerano uguali
- altrimenti si mettono in ordine per tempo.

Problema: cosa accade se arrivano altre date con tempo, per esempio `AmericanDateAndTime` e/o `AustralianDateAndTime` ?

Esempi

 Download folder

-  `AmericanDate.java`
-  `AustralianDate.java`
-  `Date.java`
-  `ItalianDate.java`
-  `ItalianDateAndTime.java`
-  `Main.java`
-  `Season.java`



Lezione del 31/10/2024, ore 16:30 - 18:30, aula Magna

Interfacce. Esempio dell'interfaccia per le date con tempo (indipendentemente dalla nazione) e suo utilizzo per semplificare i metodi `equals()` e `compareTo()` delle date. La visibilità delle interfacce e dei loro metodi è implicitamente `public`.

Esempi

Download folder

- `AmericanDate.java`
- `AmericanDateAndTime.java`
- `AustralianDate.java`
- `AustralianDateAndTime.java`
- `Date.java`
- `ItalianDate.java`
- `ItalianDateAndTime.java`
- `Main.java`
- `Season.java`
- `Time.java`



Lezione del 6/11/2024, ore 13:30 - 14:30, aula Magna

Metodi e classi **abstract**. Esempi per le figure geometriche.

Esempi

Download folder

- `Circle.java`
- `Color.java`
- `Crown.java`
- `Figure.java`
- `GreenDot.java`
- `MainFigures.java`
- `Rectangle.java`
- `Square.java`



Laboratorio dell'8/11/2024, ore 12:30 - 14:30, aula Delta

1. Si consideri la seguente interfaccia, che specifica un giocatore di calcio:

```
public interface SoccerPlayer {
    String toString(); // ritorna il nome del giocatore
    boolean canUseHands(); // determina se il giocatore può usare le mani
}
```

Si completi la seguente implementazione di **SoccerPlayer**:

```
public abstract class AbstractSoccerPlayer implements SoccerPlayer {
    ...
    protected AbstractSoccerPlayer(String name) {
        ...
    }
    public final String toString() {
        ...
    }
}
```

E' possibile aggiungere campi o metodi, ma solo **private**. Si noti che il metodo **canUseHands()** non è ancora implementato in **AbstractSoccerPlayer**, per cui tale classe deve essere **abstract**.

2. Si realizzino quattro sottoclassi concrete di **AbstractSoccerPlayer**, chiamate rispettivamente **Forward**, **Midfield**, **Defence** e **GoalKeeper**. Il costruttore di tali classi richiede il nome del giocatore come parametro. Solo il **GoalKeeper** può usare le mani nel gioco del calcio.
3. Si completi la seguente classe, che implementa una formazione del gioco del calcio, cioè l'insieme degli 11 giocatori che formano la squadra durante una partita.

```

public class Formation {
    ...
    public Formation(SoccerPlayer[] players) {
        ...
        if (!isValid())
            throw new IllegalArgumentException("invalid formation");
    }

    protected boolean isValid() {
        // ritorna true se e solo se la formazione è fatta da 11 giocatori, di cui esattamente uno è un portiere
    }

    protected SoccerPlayer[] getPlayers() {
        // ritorna i giocatori di questa formazione
    }

    public final String toString() {
        // ritorna i nomi dei giocatori della formazione, separati da virgola
    }
}

```

4. Si implementi una sottoclasse concreta di **Formation**, chiamata **Formation433**, che, per essere valida, deve essere composta da 4 difensori, 3 centrocampisti e 3 attaccanti, oltre ovviamente a un portiere. L'implementazione di **isValid()** dovrà quindi cambiare per questa sottoclasse.
5. Si scriva una classe di prova **Main** con un metodo **main()** che crea 11 giocatori: 4 difensori (Alex Sandro, Rugani, Chiellini e Dani Alves), 3 centrocampisti (Fabinho, Iniesta, Pjanic), 3 attaccanti (Dybala, Higuain, Bernardeschi) e un portiere (Szczesny). Poi crea una **Formation433**, passando tali giocatori al costruttore, e la stampa.

Soluzione

[Download folder](#)

-  AbstractSoccerPlayer.java
-  Defence.java
-  Formation.java
-  Formation433.java
-  Forward.java
-  GoalKeeper.java
-  Main.java
-  Midfield.java
-  SoccerPlayer.java



Lezione del 13/11/2024, ore 13:30 - 14:30, aula Magna

Parametri varargs. Esempio per i costruttori delle classi **Formation** e **Formation433** dell'ultimo laboratorio. Tipi generici: variabili di tipo nella dichiarazione di una classe e di una interfaccia. L'interfaccia di libreria **Comparable<T>**. Vincoli sulle variabili di tipo. Esempio di un metodo per restituire il minimo di un array non vuoto di **Comparable<T>**.

Metodi di uso frequente della classe `java.lang.Comparable<T>`

- **`int compareTo(T other)`**: ritorna un numero negativo se viene prima **this**, un numero positivo se viene prima **other** e 0 se **this** e **other** sono equivalenti.

Esempio del minimo di un array di Comparable

[Download folder](#)

- `AbstractSoccerPlayer.java`
- `Date.java`
- `Defence.java`
- `Formation.java`
- `Formation433.java`
- `Forward.java`
- `GoalKeeper.java`
- `Main.java`
- `Main2.java`
- `MainFormations.java`
- `Midfield.java`
- `Pair.java`
- `Pair1.java`
- `Pair2.java`
- `Pair3.java`
- `SoccerPlayer.java`



Lezione del 14/11/2024, ore 13:30 - 15:30, aula Magna

Classi wrapper della libreria standard. Boxing e unboxing automatico da tipi primitivi a classi wrapper e viceversa. Eccezioni. Lancio delle eccezioni implicito dal linguaggio o esplicito tramite l'istruzione `throw`. Esempio per il test di legalità delle date. Intercettazione delle eccezioni tramite il costrutto `try/catch/finally`. Gerarchia delle eccezioni. Eccezioni controllate ed eccezioni non controllate. La clausola `throws` per costruttori e metodi. Definizione di nostre classi di eccezione.

Metodi di uso frequente della classe `java.lang.Integer` (classe immutabile, nel senso che i suoi oggetti non sono più modificabili dopo la creazione)

- `static int MAX_VALUE` (costante che contiene il massimo `int` utilizzabile in Java)
- `static int MIN_VALUE` (costante che contiene il minimo `int` utilizzabile in Java)
- `Integer(int value)` (deprecato!)
- `Integer(String value)` throws `java.lang.NumberFormatException`
- `int intValue()` (restituisce il valore `int` corrispondente)
- `int compareTo(Integer other)` (infatti `Integer` implementa `Comparable<Integer>`)
- `static int parseInt(String s)` throws `java.lang.NumberFormatException` (traduce la stringa `s` in `int`)
- `static String toBinaryString(int i)` (ritorna la rappresentazione binaria di `i`)
- `static String toHexString(int i)` (ritorna la rappresentazione esadecimale di `i`)
- `static Integer valueOf(int i)` (ritorna `new Integer(i)` ma usa una cache per chiamate ripetute)

Esistono altre classi wrapper corrispondenti agli altri tipi primitivi, con costanti, costruttori e metodi simili a quanto riportato sopra: `java.lang.Short`, `java.lang.Long`, `java.lang.Float`, `java.lang.Double`, `java.lang.Byte` e `java.lang.Boolean`.

Metodi di uso frequente della classe `java.lang.Character` (classe immutabile, nel senso che i suoi oggetti non sono più modificabili dopo la creazione)

- `static char MAX_VALUE` (costante che contiene il massimo `char` utilizzabile in Java)
- `static char MIN_VALUE` (costante che contiene il minimo `char` utilizzabile in Java)
- `Character(char value)` (deprecato!)
- `char charValue()` (restituisce il valore `char` corrispondente)
- `int compareTo(Character other)` (infatti `Character` implementa `Comparable<Character>`)
- `static boolean isDigit(char c)`

- `static boolean isLetter(char c)`
- `static boolean isLetterOrDigit(char c)`
- `static boolean isLowerCase(char c)`
- `static boolean isUpperCase(char c)`
- `static boolean isWhitespace(char c)`
- `static char toLowerCase(char c)`
- `static char toUpperCase(char c)`
- `static Character valueOf(char c)` (ritorna `new Character(c)` ma usa una cache per chiamate ripetute)

Esempi

[Download folder](#)

- `AmericanDate.java`
- `AmericanDateAndTime.java`
- `AustralianDate.java`
- `AustralianDateAndTime.java`
- `Date.java`
- `IllegalDateException.java`
- `ItalianDate.java`
- `ItalianDateAndTime.java`
- `Main.java`
- `MainClass.java`
- `MainDate.java`
- `MainInteger.java`
- `MainNull.java`
- `MainPair.java`
- `Pair.java`
- `ReadString.java`
- `Season.java`
- `Time.java`



Laboratorio del 15/11/2024, ore 12:30 - 14:30, aula Delta

1. Si crei un progetto Eclipse e si copi al suo interno la seguente interfaccia, che rappresenta un numero non negativo, in una qualsiasi base di numerazione:

```
public interface Number extends Comparable<Number> {
    int getValue(); // restituisce il valore di questo numero
}
```

2. Si completi la seguente implementazione astratta di un `Number`, che fornisce le funzionalità comuni a tutti i numeri, cioè il controllo sulla non negatività del valore, l'accesso al valore, la traduzione in stringa e il metodo per il test di uguaglianza:

```
public abstract class AbstractNumber implements Number {
    private final int value;

    protected AbstractNumber(int value) {
        // se value è negativo, esegue throw new IllegalArgumentException(); altrimenti inizializza il campo value
        ...
    }

    // restituisce il valore di questo numero
    public final int getValue() { ... }

    // restituisce la base di numerazione di questo numero
    protected abstract int getBase();

    // restituisce il carattere che rappresenta la cifra "digit" nella base di numerazione
    // di questo numero. Sarà sempre vero che 0 <= digit < getBase();
    // per esempio, in base sedici si avrà getCharForDigit(10) == 'A' e
    // in base otto si avrà getCharForDigit(7) == '7'
    protected abstract char getCharForDigit(int digit);

    // restituisce una stringa che rappresenta il numero nella sua base di numerazione
    public String toString() { ... }
```

```

public final boolean equals(Object other) {
    // due numeri sono uguali se e solo se hanno lo stesso valore
    ...
}

public final int compareTo(Number other) {
    // l'ordinamento fra i Number è quello crescente per valore
    ...
}
}

```

- Si scrivano le sottoclassi concrete **DecimalNumber**, **BinaryNumber**, **OctalNumber** ed **HexNumber** di **AbstractNumber**, che rappresentano, rispettivamente, un numero in base 10, 2, 8 e 16. Queste classi si istanziano con il loro costruttore, a cui viene passato il valore del numero. Non si ridefinisca, in queste quattro sottoclassi, il metodo **toString()**: quello ereditato da **AbstractNumber** dovrà funzionare per tutte queste sottoclassi, traducendo il valore del numero nella giusta base di numerazione.
- Nella codifica binaria con parità, un numero binario viene esteso con un'ulteriore cifra binaria di controllo, in modo da rendere pari il numero totale di cifre 1. Se quindi il numero binario aveva una quantità pari di 1, si aggiungerà una cifra di controllo 0. Se invece il numero binario aveva una quantità dispari di 1, si aggiungerà una cifra di controllo 1. Questa modifica riduce il rischio di trasmissione di dati corrotti, permettendo di implementare un rudimentale sistema di rilevazione dell'errore. Si implementi una sottoclasse concreta **BinaryNumberWithParity** di **BinaryNumber**, ridefinendo solo il metodo **toString()** in modo da aggiungere in fondo la cifra di controllo opportuna.
- Nella codifica in base 58, si utilizzano 58 cifre diverse, scelte fra i numeri arabi e le lettere inglesi maiuscole e minuscole. Si evitano i caratteri **001l**, che potrebbero essere confusi a video, perché graficamente simili. Si implementi una sottoclasse concreta **Base58Number** di **AbstractNumber**, in modo da implementare questa numerazione in base 58. Le 58 cifre sono quindi **123456789ABCDEFGHJKLMNPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz**. Non si ridefinisca il metodo **toString()** ereditato da **AbstractNumber**.
- Si scriva una classe di prova **MainNumbers** con un metodo **main()** che chiede all'utente di inserire un numero non negativo n, quindi crea il numero n in base 10, poi in base 2, poi in base 2 con parità, poi in base 8, poi in base 16 e infine in base 58, stampando tutti tali numeri. Se per esempio l'utente inserisse il numero 1234567, il **main()** dovrà stampare:

```

1234567
100101101011010000111
1001011010110100001111
4553207
12D687
7Kze

```

- Si scriva una classe di prova **MainNumbersSort** con un metodo **main()** che crea un array di **Number** contenente esattamente sei elementi:
 - 2024 in base 10
 - 113 in base 2
 - 158 in base 2 con parità
 - 827 in base 8
 - 2066 in base 16
 - 8092 in base 58
Quindi ordina l'array con **java.util.Arrays.sort(...)** e lo stampa sfruttando **java.util.Arrays.toString(...)**.

Soluzione

 Download folder

-  AbstractNumber.java
-  Base58Number.java
-  BinaryNumber.java
-  BinaryNumberWithParity.java
-  DecimalNumber.java
-  HexNumber.java
-  MainNumbers.java
-  MainNumbersSort.java
-  Number.java
-  OctalNumber.java



Lezione del 20/11/2024, ore 13:30 - 14:30, aula Magna

Rendiamo `toString()` astratto nella nostra superclasse `Date`, e quindi anche la classe `Date` stessa. Aggiunta di un metodo `abstract` per la creazione di una data dello stesso tipo dell'oggetto su cui è invocato (factory method). Utilizzo in `getSeason()`. Le interfacce `java.lang.Closeable` e `java.lang.AutoCloseable`. La clausola `finally` del `try`. Il costrutto `try with resource`. Utilizzo della classe `java.util.Scanner` per semplificare la lettura di un file testuale.

Metodi di uso frequente dell'interfaccia `java.lang.AutoCloseable`

- `void close() throws java.lang.Exception`

Metodi di uso frequente dell'interfaccia `java.io.Closeable`

- `void close() throws java.io.IOException`

Esempi

 Download folder

-  AmericanDate.java
-  AmericanDateAndTime.java
-  AustralianDate.java
-  AustralianDateAndTime.java
-  Date.java
-  IllegalDateException.java
-  Input.java
-  Input2.java
-  ItalianDate.java
-  ItalianDateAndTime.java
-  Main.java
-  Season.java
-  Time.java



Lezione del 21/11/2024, ore 16:30 - 18:30, aula Magna

Le classi astratte `java.io.Reader` e `java.io.Writer` per leggere e scrivere file di caratteri. Lettura e scrittura di file di caratteri tramite le classi concrete `java.io.FileReader` e `java.io.FileWriter`. Decorazione con aggiunta di buffer tramite `java.io.BufferedReader` e `java.io.BufferedWriter`. La classe `java.io.PrintWriter` per scrivere comodamente file di caratteri. Lettura e scrittura di file di byte (file raw): classi `java.io.InputStream`, `java.io.FileInputStream`, `java.io.BufferedInputStream`, `java.io.OutputStream`, `ja-`

`va.io.FileOutputStream` e `java.io.BufferedOutputStream`. Esempio di un metodo `dumpAsText(String fileName)` aggiunto alle date, per scriverle su un file testuale, e di un costruttore che riceve uno `Scanner` per rileggere una data precedentemente scritta su file.

Metodi di uso frequente della classe astratta `java.io.Reader`

- `int read()` throws `java.io.IOException` (blocca l'esecuzione finché non c'è un carattere da leggere; a quel punto ritorna il codice Unicode del prossimo carattere letto; ritorna -1 se la sorgente di lettura è terminata)
- `int read(char[] buffer)` throws `java.io.IOException` (blocca l'esecuzione finché non arriva qualche carattere da leggere; a quel punto scrive i caratteri nel `buffer` e ritorna il numero di caratteri letti; ritorna -1 se la sorgente di lettura è terminata)

Metodi di uso frequente della classe `java.io.FileReader` (oltre quelli ereditati da `java.io.Reader`)

- `FileReader(String fileName)` throws `java.io.FileNotFoundException` (crea un lettore di file che legge i caratteri dal file di testo col nome indicato)

Metodi di uso frequente della classe `java.io.BufferedReader` (oltre quelli ereditati da `java.io.Reader`)

- `BufferedReader(Reader parent)` (crea una vista bufferizzata di `parent`)

Metodi di uso frequente della classe astratta `java.io.Writer`

- `void write(int c)` throws `java.io.IOException` (interpreta i 16 bit meno significativi di `c` come codice Unicode di un carattere e lo scrive nel file)
- `void write(char[] buffer)` throws `java.io.IOException` (scrive nel file i caratteri contenuti nell'array `buffer`)
- `void write(String s)` throws `java.io.IOException` (scrive nel file i caratteri della stringa `s`)

Metodi di uso frequente della classe `java.io.FileWriter` (oltre quelli ereditati da `java.io.Writer`)

- `FileWriter(String fileName)` throws `java.io.IOException` (crea uno scrittore di file che scrive i caratteri nel file di testo col nome indicato)

Metodi di uso frequente della classe `java.io.BufferedWriter` (oltre quelli ereditati da `java.io.Writer`)

- `BufferedWriter(Writer parent)` (crea una vista bufferizzata di `parent`)

Metodi di uso frequente della classe `java.io.PrintWriter` (oltre quelli ereditati da `java.io.Writer`)

- `PrintWriter(String fileName)` throws `java.io.FileNotFoundException` (crea uno scrittore di file che scrive i caratteri nel file di testo col nome indicato)
- `void print(int i)` (scrive i caratteri della rappresentazione decimale dell'intero `i` nel file. Questo metodo esiste anche per gli altri tipi primitivi)
- `void println(int i)` (scrive i caratteri della rappresentazione decimale dell'intero `i` nel file. Questo metodo esiste anche per gli altri tipi primitivi)
- `void print(String s)`
- `void println(String s)`
- `PrintWriter printf(String format, Object... args)` (scrive il formato nel file, in stile linguaggio C)

- 📄 Reader e writer di file di caratteri in Java
- 📄 Stream di input e output di file di byte in Java

📁 Esempi

📄 Download folder

- 📄 AmericanDate.java
- 📄 AmericanDateAndTime.java
- 📄 AustralianDate.java
- 📄 AustralianDateAndTime.java
- 📄 CopyBinaryFile.java
- 📄 CopyTextFile.java
- 📄 CopyTextFile2.java
- 📄 Date.java
- 📄 IllegalDateException.java
- 📄 ItalianDate.java
- 📄 ItalianDateAndTime.java
- 📄 Main.java
- 📄 Season.java
- 📄 Time.java
- 📄 commedia.txt



Laboratorio del 22/11/2024, ore 12:30 - 14:30, aula Delta

1. Si completi la seguente classe, che rappresenta una lista non vuota di **T**:

```
package it.univr.lists;

import java.io.IOException;

public class List<T> {
    private final T head;
    private final List<T> tail;

    // crea una lista con la testa e la coda indicate
    public List(T head, List<T> tail) {
        this.head = head;
        this.tail = tail;
    }

    // crea una lista contenente la testa indicata, seguita dagli elementi indicati
    public List(T head, T... elements) {
        ...
    }

    // restituisce una descrizione di questa lista, fatta dai toString()
    // dei suoi elementi separati da virgole
    public String toString() {
        ...
    }

    // restituisce il numero di elementi di questa lista
    public int length() {
        ...
    }

    // scrive gli elementi di questa lista (cioè il loro toString())
    // dentro il file testuale col nome indicato (un PrintWriter vi aiuterà)
    public void dump(String fileName) throws IOException {
        ...
    }
}
```

2. Si completi la seguente sottoclasse, che rappresenta una lista di interi:

```
package it.univr.lists;

import java.io.IOException;

public class IntList extends List<Integer> {
    public IntList(Integer head, IntList tail) {
        ...
    }

    public IntList(Integer head, Integer... elements) {
```

```

    } ...
    // restituisce una lista di interi letta dal file testuale indicato;
    // in caso di errore di lettura, lancia una IOException; uno Scanner vi aiuterà
    public static IntList readFrom(String fileName) throws IOException {
        ...
    }
}

```

3. Se tutto è corretto, il seguente main (già fatto, da non modificare):

```

package it.univr.lists;

import java.io.IOException;

public class Main {
    public static void main(String[] args) {
        try {
            List<String> l1 = new List<String>("hello", "how", "are", "you?");
            System.out.println(l1 + " di lunghezza " + l1.length());
            l1.dump("l1.txt");
            IntList l2 = new IntList(11, 13, 42, 9, -5, 17, 13);
            System.out.println(l2 + " di lunghezza " + l2.length());
            l2.dump("l2.txt");
            IntList l3 = IntList.readFrom("l2.txt");
            System.out.println(l3 + " di lunghezza " + l3.length());
            IntList.readFrom("l1.txt"); // fallisce perché l1.txt contiene stringhe, non interi
        }
        catch (IOException e) {
            System.out.println("Errore di I/O");
        }
    }
}

```

stamperà:

```

hello, how, are, you? di lunghezza 4
11, 13, 42, 9, -5, 17, 13 di lunghezza 7
11, 13, 42, 9, -5, 17, 13 di lunghezza 7
Errore di I/O

```

Soluzione

 Download folder

-  [IntList.java](#)
-  [List.java](#)
-  [Main.java](#)



Lezione del 4/12/2024, ore 13:30 - 14:30, aula Magna

L'interfaccia `java.lang.Iterable<E>` e suo utilizzo nel ciclo `for each`. Il framework Collections della libreria standard. Le interfacce `java.util.Collection<E>`, `java.util.List<E>`, `java.util.Queue<E>` e `java.util.Set<E>` del framework Collections. Le classi di libreria `java.util.ArrayList<E>`, `java.util.Vector<E>` (deprecata!), `java.util.LinkedList<E>` (sia lista che coda FIFO) e `java.util.PriorityQueue<E>` (coda a priorità). Costruzione di una lista con i factory method.

Metodi di uso frequente dell'interfaccia `java.util.Collection<E>`

- `boolean add(E element)` (ritorna true se l'elemento viene aggiunto)
- `boolean addAll(Collection<E> other)` (ritorna true se almeno un elemento viene aggiunto)
- `boolean contains(Object element)`
- `boolean containsAll(Collection<?> other)`
- `boolean isEmpty()`
- `boolean remove(Object element)` (ritorna true se l'elemento viene rimosso)
- `boolean removeAll(Collection<?> other)` (ritorna true se almeno un elemento viene rimosso)
- `boolean retainAll(Collection<?> other)` (ritorna true se almeno un elemento viene rimosso)
- `int size()`

Metodi di uso frequente dell'interfaccia `java.util.List<E>` (oltre a quelli ereditati da `java.util.Collection<E>`)

- `boolean add(E element)` (aggiunge `element` in fondo alla lista, anche se la lista già lo conteneva; ritorna sempre `true`)
- `void add(int index, E element)` (piazza l'elemento alla posizione `index`, che deve essere fra 0 e `size()` inclusi, spostando di una posizione a destra l'elemento che c'era precedentemente e quelli alla sua destra)
- `E get(int index)` (ritorna l'elemento alla posizione `index`, che deve essere fra 0 incluso e `size()` escluso)
- `int indexOf(Object element)` (ritorna la prima posizione in cui occorre `element`; ritorna -1 se la lista non contiene `element`)
- `static <E> List<E> of(E... elements)` (factory method che costruisce una lista immutabile con dentro `elements`)
- `boolean remove(Object element)` (rimuove la prima occorrenza di `element`, se presente; ritorna `true` se l'elemento viene rimosso)
- `E remove(int index)` (rimuove e ritorna l'elemento alla posizione `index`, che deve essere fra 0 incluso e `size()` escluso; gli elementi alla sua destra vengono spostati di una posizione a sinistra)
- `E set(int index, E element)` (ritorna l'elemento alla posizione `index`, che deve essere fra 0 e `size()` inclusi, e lo sostituisce con `element`)

Metodi di uso frequente dell'interfaccia `java.util.Queue<E>` (oltre a quelli ereditati da `java.util.Collection<E>`)

- `E poll()` (rimuove e ritorna la testa della coda; ritorna `null` se la coda è vuota)
- `E remove() throws java.util.NoSuchElementException` (rimuove e ritorna la testa della coda, se non è vuota; altrimenti lancia un'eccezione)
- `E peek()` (ritorna la testa della coda, senza rimuoverla; ritorna `null` se la coda è vuota)
- `E element() throws java.util.NoSuchElementException` (ritorna la testa della coda, senza rimuoverla; se la coda è vuota, lancia un'eccezione)
- `boolean offer(E element)` (aggiunge `element` in fondo alla coda, se c'è spazio. Ritorna `true` se e solo se l'elemento viene aggiunto)
- `boolean add(E element) throws java.lang.IllegalStateException` (aggiunge `element` in fondo alla coda, se c'è spazio, altrimenti lancia un'eccezione. Ritorna sempre `true`)

Metodi di uso frequente dell'interfaccia `java.util.Set<E>` (oltre a quelli ereditati da `java.util.Collection<E>`)

- `static <E> Set<E> of(E... elements)` (factory method)

Metodi di uso frequente della classe `java.util.LinkedList<E>`

- `LinkedList()`
- `LinkedList(Collection<? extends E> parent)` (crea una lista e la riempie con gli elementi di `parent`)

Metodi di uso frequente della classe `java.util.ArrayList<E>`

- `ArrayList()`
- `ArrayList(Collection<? extends E> parent)` (crea una lista e la riempie con gli elementi di `parent`)

Metodi di uso frequente della classe `java.util.PriorityQueue<E>` (oltre a quelli ereditati da `java.util.Queue<E>`). Si tratta di una coda **unbounded**: non ha un limite massimo di elementi ma si espande quando necessario.

- `PriorityQueue()`
- `PriorityQueue(Collection<? extends E> parent)` (crea una coda e la riempie con gli elementi di `parent`)

Gerarchia delle collezioni

Esempi

[Download folder](#)

- [MainArrayList.java](#)
- [MainLinkedList.java](#)
- [MainPriorityQueue.java](#)
- [MainPriorityQueue2.java](#)

Lezione del 5/12/2024, ore 16:30 - 18:30, aula Magna

Classe di libreria `java.util.HashSet<E>`. Il metodo `equals()` deve essere una relazione di equivalenza (riflessiva, transitiva e simmetrica). Metodo `hashCode()` e sua consistenza con il metodo `equals(Object)`. Metodo `hashCode()` non banale. Interfaccia del framework Collections: `java.util.SortedSet<E>`. La classe di libreria `java.util.TreeSet<E>`. Consistenza fra `compareTo()` ed `equals()`. Iterazione su una collezione: iteratori fail-safe e `ConcurrentModificationException`. Esempio: implementiamo `compareTo()`, `equals()` ed `hashCode()` sulle `Dates`, poi modifichiamo un insieme di `Date` eliminando le date in autunno. L'interfaccia `java.util.Map<K,V>` e la classe concreta di libreria `java.util.HashMap<K,V>`. Esempio di utilizzo delle mappe per implementare un dizionario. L'interfaccia `java.util.SortedMap<K,V>` e la classe concreta di libreria `java.util.TreeMap<K,V>`. Possibilità di creare insieme ordinati specificando un ordinamento degli elementi diverso da quello naturale.

Metodi di uso frequente della classe `java.util.HashSet<E>` (oltre a quelli ereditati da `java.util.Set<E>`)

- `HashSet()`
- `HashSet(Collection<? extends E> parent)` (crea un insieme e lo riempie con gli elementi di `parent`)

Metodi di uso frequente dell'interfaccia `java.util.SortedSet<E>` (oltre a quelli ereditati o ridefiniti da `java.util.Set<E>`)

- `E first() throws java.util.NoSuchElementException` (ritorna, ma non rimuove, l'elemento più piccolo dell'insieme)
- `E last() throws java.util.NoSuchElementException` (ritorna, ma non rimuove, l'elemento più grande dell'insieme)

Metodi di uso frequente della classe `java.util.TreeSet<E>` (oltre a quelli ereditati o ridefiniti da `java.util.SortedSet<E>`)

- `TreeSet()`
- `TreeSet(Collection<? extends E> parent)` (crea un insieme ordinato e lo riempie con gli elementi di `parent`)

Metodi di uso frequente dell'interfaccia `java.util.Map<K,V>`

- `boolean containsKey(Object key)`
- `boolean containsValue(Object value)`
- `V get(Object key)`
- `V getOrDefault(Object key, V defaultValue)`
- `boolean isEmpty()`
- `Set<K> keySet()`
- `V put(K key, V value)` (setta il valore per la chiave; rimpiazza il valore se la chiave era già presente; ritorna il vecchio valore, se già presente, altrimenti `null`)
- `V putIfAbsent(K key, V value)` (rimpiazzia il valore ma solo se la chiave non era già presente; ritorna il vecchio valore, se già presente, altrimenti `null`)
- `V remove(Object key)` (ritorna il vecchio valore, se era presente; altrimenti `null`)
- `int size()`
- `Collection<V> values()`

Metodi di uso frequente della classe `java.util.HashMap<K,V>`

- `HashMap()`
- `HashMap(Map<? extends K,? extends V> parent)` (crea una mappa e la riempie con le coppie contenute in `parent`)

Metodi di uso frequente dell'interfaccia `java.util.SortedMap<K,V>` (oltre a tutti quelli di `Map`)

- `K firstKey()`
- `K lastKey()`

Metodi di uso frequente della classe `java.util.TreeMap<K,V>`

- `TreeMap()`
- `TreeMap(Map<? extends K,? extends V> parent)` (crea una mappa e la riempie con le coppie contenute in `parent`)

Esempi

[Download folder](#)

- `ComparatorByLength.java`
- `Date.java`
- `Dictionary.java`
- `Dictionary2.java`
- `MainHashSet.java`
- `MainHashSet2.java`
- `MainHashSetModification.java`
- `MainTreeSet.java`
- `MainTreeSet2.java`
- `MainTreeSet3.java`



Laboratorio del 6/12/2024, ore 12:30 - 14:30, aula Delta

Si crei un progetto con le classi allegate sotto, che avevamo scritto per implementare delle carte da poker. Le si inserisce dentro un package `it.univr.cards`. Quindi le si modifichi come segue.

- Si trasformi il metodo `equals()` di `Card` in quello atteso dalla libreria Collections, cioè in `boolean equals(Object other)`.
- Si aggiunga alla classe `Card` un metodo `int hashCode()` non banale.
- Si faccia implementare a `Card` l'interfaccia di libreria `Comparable<Card>`, in modo che le carte vengano ordinate prima per valore crescente e, a parità di valore, per seme crescente.
- Si scriva una classe di prova `MainCards`, dentro il package `it.univr.cards`, che crea un `HashSet<Card>` e un `TreeSet<Card>`, inizialmente vuoti. Poi riempie l'`HashSet` con 100 milioni di carte casuali e stampa il tempo richiesto per tale operazione (si sfrutti il metodo `System.currentTimeMillis()`). Successivamente riempie il `TreeSet` con 100 milioni di carte casuali e stampa il tempo richiesto per tale operazione. Infine stampa la dimensione dell'`HashSet`, la dimensione del `TreeSet` e l'`equals()` fra l'`HashSet` e il `TreeSet`. È più veloce l'`HashSet` o il `TreeSet`? I due insiemi risultano `equals()`?
- Si modifichi l'`hashCode()` della classe `Card` in modo da renderlo banale (fategli ritornare una costante). Adesso è più veloce l'`HashSet` o il `TreeSet`?
- Si crei una classe `Deck` che implementa un mazzo di carte da poker, tutte diverse, completando il seguente codice (altri metodi e i campi devono essere dichiarati `private`)

```
package it.univr.cards;

import java.util.SortedSet;
import java.util.TreeSet;

public class Deck {
    ...

    /**
     * Costruisce un mazzo di size carte, tutte diverse, il cui valore è da min in su.
     */
    public Deck(int size, Value min) { ... }
```

```
/**
 * Restituisce una stringa che descrive le carte in questo mazzo, costruita come la loro stampa in sequenza
 * crescente per valore, separate da virgola, con parentesi quadre agli estremi.
 */
public String toString() { ... }

/**
 * Restituisce la carta più alta di questo mazzo (senza rimuoverla).
 */
public Card best() { ... }

/**
 * Rimuove la carta più alta di questo mazzo.
 */
public void removeBest() { ... }
}
```

- Si verifichi che l'esecuzione di **MainDeck** stampa qualcosa del tipo:

```
deck1: ♠7, ♦10, ♠J, ♠Q, ♠K]
best of deck1: ♠K
deck1 without ♠K: ♠7, ♦10, ♠J, ♠Q]

deck2: ♠5, ♠8, ♠9, ♠J, ♠1]
best of deck2: ♠1

deck3: ♠2, ♠2, ♠2, ♥2, ♠3, ♠3, ♠3, ♥3, ♠4, ♠4, ♠4, ... , ♠K, ♠K, ♠K, ♥K, ♠1, ♠1, ♠1, ♥1]
best of deck3: ♥1
```

File da cui iniziare le modifiche

[Download folder](#)

- [Card.java](#)
- [MainDeck.java](#)
- [Suit.java](#)
- [Value.java](#)

Soluzione

[Download folder](#)

- [Card.java](#)
- [Deck.java](#)
- [MainCards.java](#)
- [MainDeck.java](#)
- [Suit.java](#)
- [Value.java](#)

Lezione dell'11/12/2024, ore 13:30 - 14:30, aula Magna

Le interfacce di libreria **java.lang.Iterable<T>** e **java.util.Iterator<T>**. Loro implementazione per rendere iterabili le nostre classi. Esempio di implementazione di **iterator()** per delega, per il caso del **Desk** del laboratorio del 6 dicembre. Classi interne e classi interne statiche. Esempio di implementazione con una classe interna.

Metodi di uso frequente dell'interfaccia **java.lang.Iterable<T>**

- **Iterator<T> iterator()** (ritorna un iteratore, cioè un oggetto capace di restituire gli elementi dell'iterabile, uno alla volta)

Metodi di uso frequente dell'interfaccia **java.util.Iterator<T>**

- **boolean hasNext()**
- **T next()**

Iterabile e Iteratore

Esempi

[Download folder](#)

- [Chat.java](#)
- [Deck.java](#)
- [MainDeck.java](#)



Lezione del 12/12/2024, ore 16:30 - 18:30, aula Magna

Esempio dell'iterabile delle sequenze lessicografiche. Concorrenza. La classe `java.lang.Thread`. Race condition. Deadlock. Il costrutto `synchronized`. Classi e oggetti immutabili. Esempio del calcolo delle auto-frasi in maniera sequenziale e parallela.

Metodi di uso frequente della classe `java.lang.Thread`

- `Thread()` (costruisce un thread che, quando verrà mandato in esecuzione, eseguirà il codice del suo metodo `run()`)
- `Thread(Runnable runnable)` (costruisce un thread che, quando verrà mandato in esecuzione, eseguirà il codice del metodo `run()` del `runnable`)
- `void start()` (manda in esecuzione il metodo `run()` del thread e ritorna immediatamente il controllo al chiamante; i due thread continueranno in parallelo)
- `void run()` (specifica il codice che viene eseguito in parallelo quando si chiama `start()`)
- `void join() throws java.lang.InterruptedException` (sospende il chiamante finché il thread destinatario non termina; se il chiamante viene interrotto mentre aspetta, viene lanciata una `InterruptedException`)
- `static void sleep(long milliseconds) throws InterruptedException` (sospende il chiamante per il numero di millisecondi indicato; se il chiamante viene interrotto mentre aspetta, viene lanciata una `InterruptedException`)
- `static Thread currentThread()` (ritorna il thread che sta eseguendo il codice chiamante)

[Constants.java](#)

Esempi

[Download folder](#)

- [AutoSentences.java](#)
- [AutoSentencesThread.java](#)
- [Constants.java](#)
- [Sentence.java](#)
- [Sequences.java](#)



Laboratorio del 13/12/2024, ore 12:30 - 14:30, aula Delta

[Compito del 18-2-2019.pdf](#)

File di partenza

Download folder

- [AudioBook.java](#)
- [Book.java](#)
- [Catalog.java](#)
- [CatalogWithStatistics.java](#)
- [Genre.java](#)
- [Main.java](#)
- [PaperBook.java](#)
- [risultato_main.txt](#)

Soluzione

Download folder

- [AudioBook.java](#)
- [Book.java](#)
- [ByAuthor.java](#)
- [ByYear.java](#)
- [Catalog.java](#)
- [CatalogWithStatistics.java](#)
- [Genre.java](#)
- [Main.java](#)
- [PaperBook.java](#)



Lezione del 18/12/2024, ore 13:30 - 14:30, aula Magna

Classi anonime. Lambda espressioni. Interfacce funzionali. Riferimenti a metodi e costruttori. Uso delle lambda espressioni per iterazione interna su collezioni.

Metodi di uso frequente dell'interfaccia [java.util.function.Consumer<T>](#)

- `void accept(T t)` (esegue del codice che usa `t`)

Metodi di uso frequente dell'interfaccia [java.util.function.Predicate<T>](#)

- `boolean test(T t)` (determina se `t` soddisfa il predicato)

Metodi di uso frequente dell'interfaccia [java.util.function.Supplier<T>](#)

- `T get()` (fornisce un oggetto di tipo `T`)

Metodi di uso frequente dell'interfaccia [java.util.function.Function<T,U>](#)

- `boolean U apply(T t)` (ritorna il valore della funzione applicata a `t`)

[Slide sulle lambda espressioni e gli stream](#)

Esempi

Download folder

- [MainLists.java](#)



Laboratorio del 18/12/2024, ore 16:30 - 18:30, aula Delta

1. Si completi la seguente classe, che implementa un calendario annuale. Si consideri il caso degli anni bisestili. E' possibile aggiungere campi, costruttori e metodi ma solo `private`.

```

public class Calendar {
    private final int year; // l'anno del calendario

    public Calendar(int year) {
        // costruisce il calendario per l'anno indicato
    }

    public boolean isLeapYear() {
        // determina se il calendario è per un anno bisestile
    }

    public Date getStart() {
        // restituisce la prima data del calendario (primo gennaio)
    }

    public Date getEnd() {
        // restituisce l'ultima data del calendario (31 dicembre)
    }

    // classe interna
    public class Date {
        // 0 = primo gennaio, 364 = 31 dicembre (per i bisestili: 365 = 31 dicembre)
        private final int daysFromStart;

        private Date(int daysFromStart) {
            this.daysFromStart = daysFromStart;
        }

        // ritorna il giorno di questa data, tra 1 e 31
        public int getDay() {
            ...
        }

        // ritorna il mese di questa data tra 1 e 12
        public int getMonth() {
            ...
        }

        // ritorna l'anno di questa data
        public int getYear() {
            ...
        }

        public String toString() {
            return String.format("%d/%d/%d", getDay(), getMonth(), getYear());
        }
    }
}

```

2. Si renda **Calendar** iterabile, dichiarandolo come **implements Iterable<Calendar.Date>**, in modo che iterando su un calendario si ottengano le date dal primo gennaio al 31 dicembre dell'anno del calendario.

Si usi il seguente main di prova, che dovrebbe stampare prima e ultima data del 2024 e tutte le date del 2024, dal primo gennaio al 31 dicembre:

```

public class MainCalendar {

    public static void main(String[] args) {
        Calendar cal = new Calendar(2024);
        System.out.println("start: " + cal.getStart());
        System.out.println("end: " + cal.getEnd());
        for (Calendar.Date date: cal)
            System.out.println(date);
    }
}

```

Soluzione

[Download folder](#)

-  [Calendar.java](#)
-  [MainCalendar.java](#)



Lezione del 19/12/2024, ore 16:30 - 18:30, aula Magna

Gli stream lazy. Schema di programmazione create/filter/map/reduce. Classi **IntStream**, **LongStream** e **Stream**. Funzioni sugli stream: **filter**, **map**, **flatMap**, **distinct**, **limit**, **sorted**, **anyMatch**, **allMatch**, **noneMatch**, **forEach**, **count**, **findAny** e **findFirst**. Gli stream paralleli.

Esempi

[Download folder](#)

- [AutoSentencesStream.java](#)
- [ManyAs.java](#)



Laboratorio del 20/12/2024, ore 12:30 - 14:30, aula Delta

1. Si completi la seguente classe **Board**, che implementa una tavola bidimensionale di cellule, rappresentate come booleani. Il valore `true` indica che la cellula è viva, mentre il valore `false` indica che essa è morta. Il metodo `toString()` (già scritto) riporta le cellule vive con un asterisco e quelle morte con uno spazio.

```
public class Board {
    private final int width;
    private final int height;
    private boolean[][] current;
    ...

    /**
     * Inizializza la tavola alle dimensioni indicate, in modo che
     * contenga esattamente howManyAlive cellule vive, posizionate
     * casualmente nella tavola.
     *
     * @param width la larghezza della tavola
     * @param height l'altezza della tavola
     * @param howManyAlive il numero di cellule vive della tavola
     * @throws IllegalArgumentException se width o height sono negative,
     * oppure se howManyAlive è negativo o maggiore del numero di cellule della tavola
     */
    public Board(int width, int height, int howManyAlive) {
        ...
        this.width = width;
        this.height = height;
        this.current = new boolean[width][height];
        ...
    }

    public int getWidth() { ... }

    public int getHeight() { ... }

    /**
     * Determina se la cellula (x,y) è viva o morta.
     */
    public boolean isAliveAt(int x, int y) { ... }

    public String toString() {
        String result = "";
        for (int y = 0; y < height; y++) {
            for (int x = 0; x < width; x++) {
                result += isAliveAt(x, y) ? '*' : ' ';
            }
            result += '\n';
        }
        return result;
    }

    /**
     * Stampa ripetutamente questa tavola, chiamando ogni volta
     * next() per farla passare alla generazione successiva. L'unico modo
     * per terminare questo metodo è con CTRL+C. In Eclipse, tale segnale
     * viene inviato cliccando sul pulsante quadrato rosso della console.
     *
     * @param processor la specifica di come si passa da una generazione alla successiva
     */
    public void play(NextAliveProcessor processor) throws InterruptedException {
        while (true) {
            System.out.println(this);
            System.out.println("-----");
            next(processor);
            Thread.sleep(500); // aspetta mezzo secondo
        }
    }

    /**
     * Modifica la tavola in modo che la cellula (x,y) sarà viva
     * se e solo se processor.isAliveNextAt(x,y) è vero.
     */
    private void next(NextAliveProcessor processor) { ... }
}
```

La classe **Board** definisce un metodo privato `next()` che modifica le cellule facendole transire dalla generazione corrente a quella successiva: il modo in cui si calcola la generazione successiva da quella corrente è specificato tramite un oggetto che implementa la seguente interfaccia:

```
public interface NextAliveProcessor {
    boolean isAliveNextAt(int x, int y);
}
```

2. Si crei una classe di prova **MainAtLeast3** il cui **main** crea una **Board** di dimensione 40x20 con esattamente 100 cellule vive posizionate a caso, quindi chiama il metodo **play** su tale **Board**, con un processore che rende una cellula viva alla prossima generazione, se e solo se nella generazione corrente essa ha almeno 3 cellule adiacenti vive. **Suggerimento:** definite una classe che implementa **NextAliveProcessor** e che riceve la **Board** tramite il suo costruttore. In questo modo il processore può controllare lo stato delle cellule adiacenti.
3. Si crei una classe di prova **MainGameOfLife** identica a **MainAtLeast3**, ma che usa un processore che rende una cellula viva alla prossima generazione basandosi sulle **quattro regole del gioco della vita**.
4. Si crei una classe di prova **MainConstant** identica a **MainAtLeast3**, ma che usa un processore che rende una cellula viva alla prossima generazione se e solo se essa è viva nella generazione corrente (quindi il gioco non evolverà).
5. Si reimplementino i punti 2-3-4 usando delle lambda espressioni come processori.

 Vicini

 Soluzione

 Download folder

-  [Board.java](#)
-  [MainAtLeast3.java](#)
-  [MainAtLeast3Lambda.java](#)
-  [MainConstant.java](#)
-  [MainConstantLambda.java](#)
-  [MainGameOfLife.java](#)
-  [MainGameOfLifeLambda.java](#)
-  [NextAliveProcessor.java](#)

 [Link alla registrazione della soluzione](#)



Laboratorio del 10/1/2025, ore 12:30 - 14:30, aula Delta

Si realizzi il seguente compito, partendo dalle classi parzialmente già scritte.

 Consegna del compito

 Download folder

-  [compito.pdf](#)
-  [Emoji.java](#)
-  [Main.java](#)
-  [Sudoku.java](#)

 Soluzione

 Download folder

-  [Emoji.java](#)
-  [Main.java](#)
-  [Sudoku.java](#)



Laboratorio del 17/1/2025, ore 12:30 - 14:30, aula Delta

Si realizzi il seguente compito, partendo dalle classi parzialmente già scritte.

Consegna del compito

 Download folder

-  compito.pdf
-  Elezioni.java
-  ElezioniVincitore.java
-  Main.java
-  Partito.java
-  VotiPerPartito.java



Laboratorio del 24/1/2025, ore 12:30 - 14:30, aula Delta

Si realizzi il seguente compito, partendo dalle classi parzialmente già scritte.

Consegna del compito

 Download folder

-  compito.pdf
-  Corso.java
-  Esame.java
-  MainEsame.java
-  Studente.java
-  StudenteLavoratore.java



Primo Appello di Programmazione 2 del 5/2/2025, aula Delta

L'esame consiste nella scrittura di programmi al calcolatore del laboratorio Delta. E' possibile consultare libri e appunti. Non sarà disponibile internet e non si potranno usare computer personale, tablet o cellulari.

I turni saranno i seguenti:

- primo turno ore 9-11: studenti con matricola minore o uguale di VR500000
- secondo turno ore 11:30-13:30: studenti con matricola tra VR500001 e VR501425 inclusi
- terzo turno ore 14:30-16:30: studenti con matricola maggiore o uguale di VR501426



Secondo Appello di Programmazione 2 del 20/2/2025, aula Delta

L'esame consiste nella scrittura di programmi al calcolatore del laboratorio Delta. E' possibile consultare libri e appunti. Non sarà disponibile internet e non si potranno usare computer personale, tablet o cellulari.

I turni saranno i seguenti:

- primo turno ore 9-11
- secondo turno ore 11:30-13:30
- terzo turno ore 14:00-16:00